

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Student Name: T. Akshaya

Roll No.: A24126552122

1. TITLE

Implement Modules in Node.js

2. AIM

To understand and implement different types of modules in Node.js — local modules, built-in modules, and third-party modules.

3. OBJECTIVE

- To understand the concept of modules in Node.js.
 - To create a local module and export functions from it using `module.exports`.
 - To import and use a local module in another file using `require()`.
 - To use built-in Node.js modules such as `fs` and `os`.
 - To install and use a third-party module (Express) to create a simple web server.
-

4. THEORY

A module in Node.js is a reusable block of code that can be exported from one file and used in another. Node.js follows the CommonJS module system, where `module.exports` is used to expose functions or values from a file, and `require()` is used to import them wherever they are needed. Modules in Node.js are of three types.

Local modules are files created by the developer within the project, such as `node.js` in this program, which exports the `add` and `subtract` functions. Built-in (core) modules come pre-installed with Node.js and do not need to be downloaded separately — examples include `fs`, which is used to read and write files, and `os`, which provides information about the operating system, such as its platform and total memory.

Third-party modules are packages published by other developers and installed using `npm` (Node Package Manager), such as `Express`, which is a lightweight framework used to build web servers. It provides methods like `app.get()` to define routes and `app.listen()` to start the server on a given port.

5. ALGORITHM / PROCEDURE

1. Create a local module file (`node.js`) with `add` and `subtract` functions.
2. Export these functions using `module.exports`.

3. Create a main file (exp5_1.js) and import the local module using require().
 4. Call the imported add function and print the result.
 5. Import the built-in fs module.
 6. Use fs.writeFileSync() to create a text file with some content.
 7. Use fs.readFileSync() to read the content back and print it.
 8. Import the built-in os module.
 9. Print the operating system platform using os.platform().
 10. Print the total system memory using os.totalmem().
 11. Install Express using npm install express.
 12. Import Express and create an application using express().
 13. Define a GET route for the home page that sends a response.
 14. Start the server using app.listen() on port 3000.
 15. Run the file using Node.js and check the console output and the browser response.
 16. Test each type of module and record the results as test cases.
-

6. PROGRAM

node.js (Local Module)

```
function add(a, b) {  
  return a + b;  
}  
function subtract(a, b) {  
  return a - b;  
}  
// Export multiple things  
module.exports = { add, subtract };
```

exp5_1.js (Main File)

```
//LOCAL module  
const { add, subtract } = require('./node.js');  
console.log(add(2, 3));  
  
//BUILD IN module  
//fs-read,write  
const fs = require('fs');  
fs.writeFileSync('hello.txt', 'Hello World!');  
const content = fs.readFileSync('hello.txt','utf8');  
console.log(content);  
  
//os-operating system info  
const os = require('os');  
console.log(os.platform());  
console.log(os.totalmem());  
  
//third party module  
// express  
const express = require('express');  
const app = express();  
app.get('/', (req, res) => res.send('Hello!'));
```

```
app.listen(3000);
```

7. CODE EXPLANATION

Code	Purpose
module.exports = { add, subtract };	Exports the add and subtract functions from node.js so they can be used in other files.
require('./node.js')	Imports the local module and gives access to its exported functions.
require('fs')	Imports the built-in File System module, used to read and write files.
fs.writeFileSync(file, data)	Creates a file (or overwrites it) and writes the given text into it, synchronously.
fs.readFileSync(file, 'utf8')	Reads the content of a file as plain text and returns it.
require('os')	Imports the built-in Operating System module, used to get system information.
os.platform()	Returns the operating system platform the code is running on (e.g. linux, win32).
os.totalmem()	Returns the total system memory (RAM) in bytes.
require('express')	Imports the third-party Express module (installed via npm) used to build a web server.
app.get('/', callback)	Defines a route that runs the callback and sends a response when the home page ("/") is requested.
app.listen(3000)	Starts the Express server so it listens for requests on port 3000.

8. TEST CASES AND EXECUTION DATA

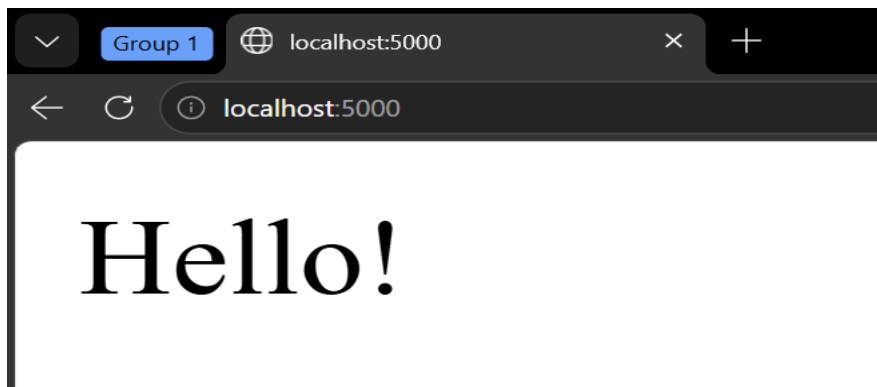
Test Case	Action	Expected Result	Actual Result	Status
TC-01	Run add(2, 3) from the local module	5	5	Pass
TC-02	Write and read hello.txt using fs	"Hello World!" is written and read back	Hello World!	Pass
TC-03	Print the OS platform using os.platform()	Shows the current OS platform	Win32	Pass
TC-04	Print total memory using os.totalmem()	Shows total system memory in bytes	16859361280	Pass
TC-05	Run the Express server and open http://localhost:3000	Browser displays "Hello!"	Page displayed "Hello!" correctly	Pass

9. OUTPUT

Output 1: Console Output on Running exp5_1.js

```
Problems Output Debug Console Terminal Ports
PS C:\Users\syama\OneDrive\Desktop\122> node exp5_1.js
5
Hello World!
win32
16859361280
node:events:505
  throw er; // Unhandled 'error' event
  ^
Error: listen EADDRINUSE: address already in use :::5000
```

Output 2: Browser Output at <http://localhost:5000>



10. RESULT

Thus, the different types of modules in Node.js — a local module, built-in modules (fs and os), and a third-party module (Express) — were successfully implemented and tested. The program correctly imported and used functions and data from each module type, and the output was verified.